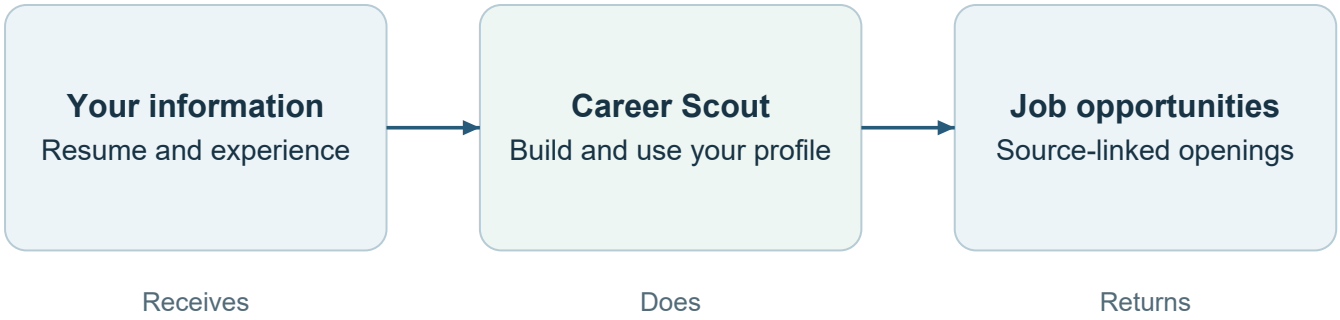
Career Scout

A visual guide to the project

Agents, user workflow, data storage and external services



Career Scout records evidence from your resume, helps you add relevant experience and uses the resulting profile to find actual job openings.

JD means job description.

01 Agent communication hierarchy 2

02 Role of each agent 3–9

03 Complete user workflow 10

04 Database and storage design 11

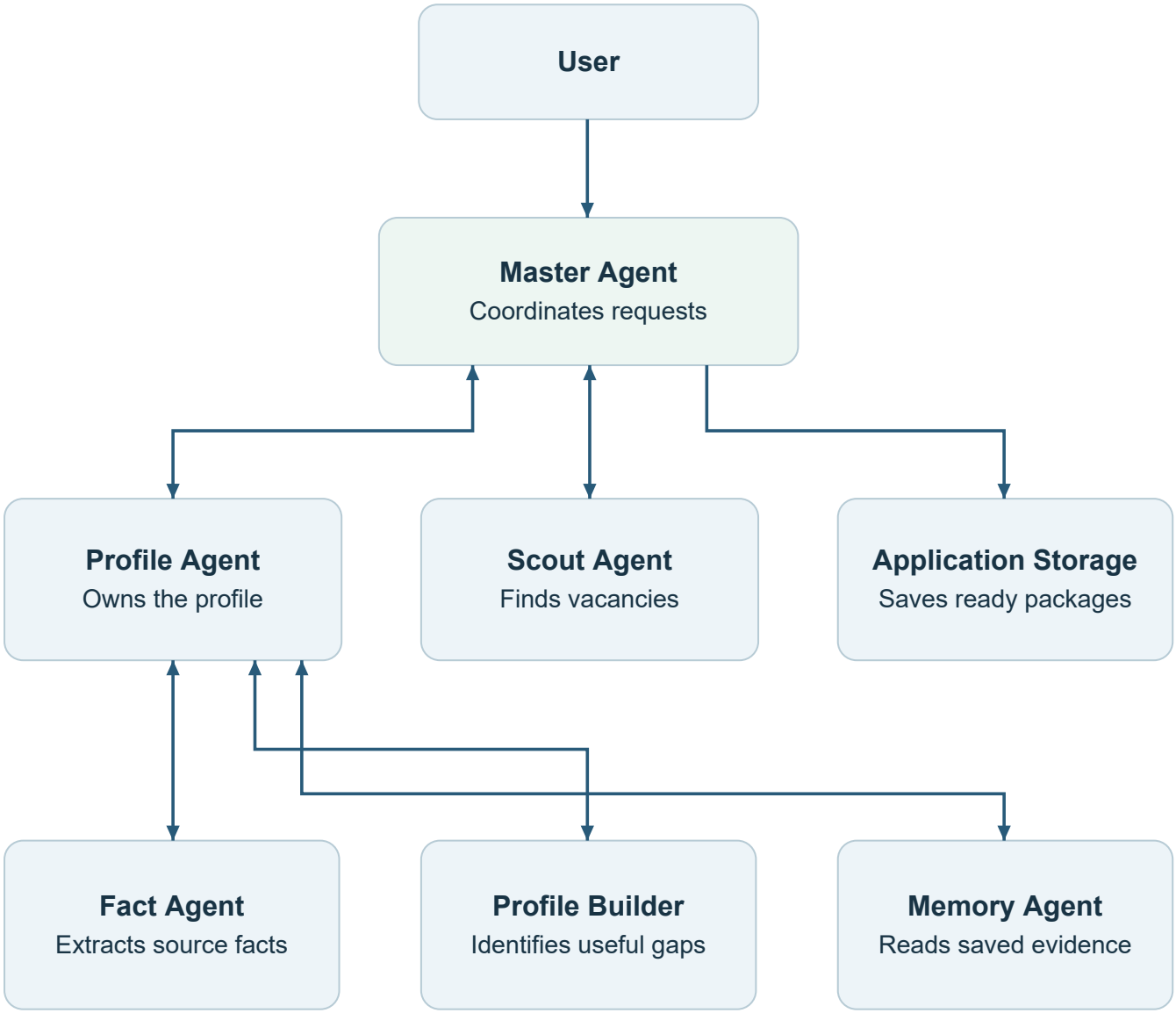
05 Use of external tools and APIs 12–13

10 November 2026 • Current local Career Scout implementation

01 AGENT COMMUNICATION HIERARCHY

How the agents communicate

Master coordinates the main workflow. Profile coordinates profile work.



Arrows show permitted requests. Double-ended arrows also show results returning through the same coordinator.

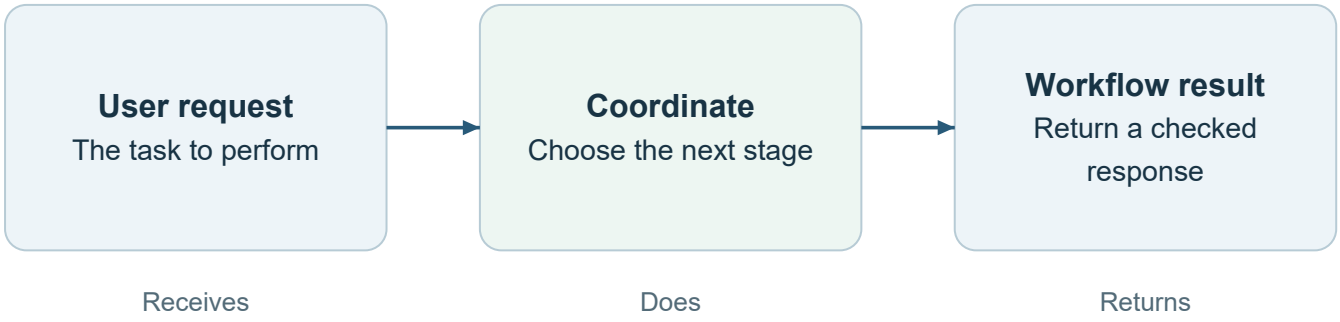
Two coordination rules

Master sends requests to Profile and Scout. **Profile** alone calls Fact, Profile Builder and Memory. Specialists return their results through the same coordinator.

Application Storage receives an authorized save command from Master and returns a storage receipt.

Master Agent

The coordinator for the overall workflow



Purpose

Master keeps the agents working in the correct order and ensures each one receives the information it needs.

What it does

Loads the current profile through Profile. Sends eligible, scoped search context to Scout when the user starts a search. Checks the selected profile version and routes storage requests only when the required checks have passed.

What it produces

Task instructions, workflow status, agent handoffs and save commands. Its routing decisions are enforced by code; changing a prompt does not grant new permissions.

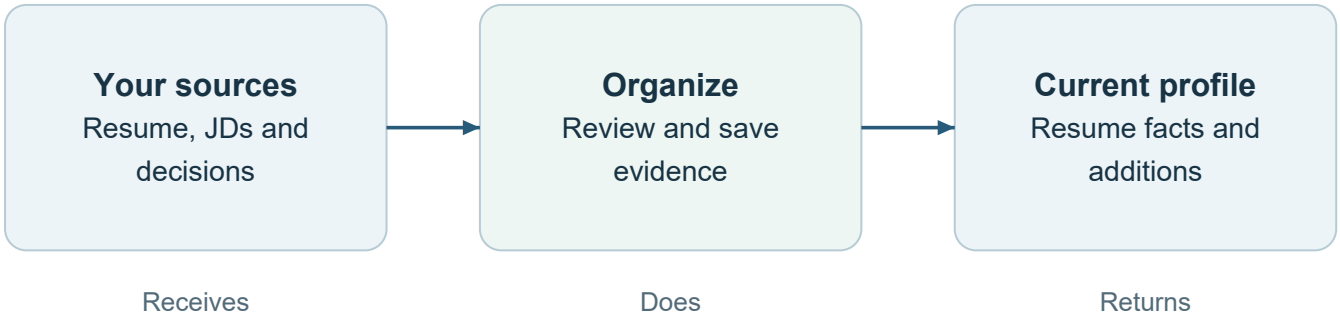
Who it works with

Works with Profile, Scout and Application Storage within the scope shown in this guide. It reaches Fact, Profile Builder and Memory through Profile.

User → Master → Profile or Scout → Master → User

Profile Agent

The coordinator for your professional profile



Purpose

Profile manages the saved information used to understand your professional experience.

What it does

Receives documents and calls Fact to extract evidence. Calls Profile Builder to compare job requirements. Saves exact user-approved additions, then reloads storage to verify that the save succeeded.

What it produces

A current profile containing eligible resume information, approved additions, source references and preferences. A separate Final Profile save is not required in the current website.

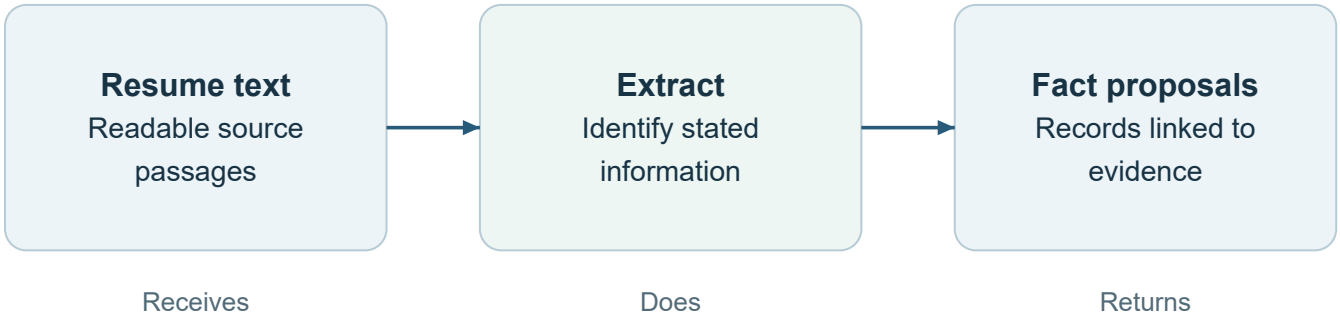
Who it works with

Reports to Master and coordinates Fact, Profile Builder and Memory. User confirmation is kept separate from information that is merely stated in a resume.

Master ↔ Profile ↔ Fact, Profile Builder and Memory

Fact Agent

Turns source text into factual records



Purpose

Fact Agent identifies what the supplied resume actually says and preserves the evidence behind each record.

What it does

Extracts employment, education, certifications, skills and professional experience. Keeps names, roles, dates and metrics tied to their source passages. Supporting code calculates employment duration without double-counting overlapping periods.

What it produces

Structured factual proposals and exact source references. Missing or ambiguous information remains unresolved. Extraction includes deterministic parsing and Gemini-assisted analysis where requested.

Who it works with

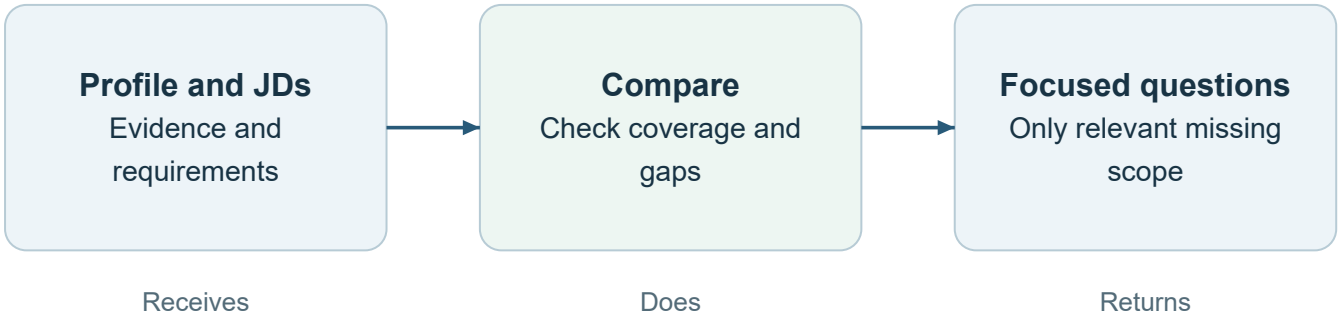
Receives tasks from Profile and returns proposals to Profile. It does not approve its own output, invent missing dates or use an employer’s JD as evidence of the user’s experience.

Profile → Fact Agent → Profile → User review

02 ROLE OF EACH AGENT

Profile Builder

Compares job requirements with your profile



Purpose

Profile Builder checks whether the saved profile already supports a job requirement before asking about additional experience.

What it does

Reads selected JDs, checks source-backed work and skills, and groups questions by role and professional topic. A requirement already covered by valid evidence is shown as recorded. Proposed questions receive a separate evidence review.

What it produces

Recorded matches, focused questions, minor gaps that can be skipped, and unresolved comparisons. Approve saves exact wording; Reject adds no claim; Somewhat saves a draft that needs approval of the revised wording.

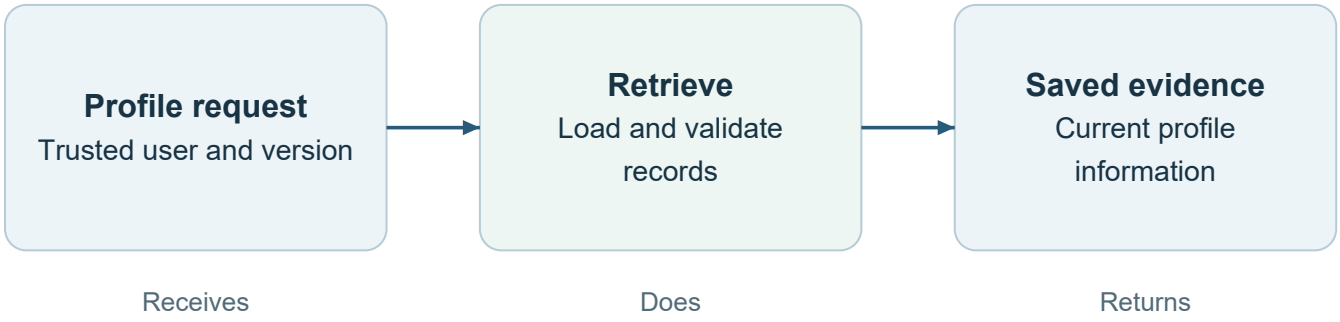
Who it works with

Works through Profile, which owns all saves and approvals. The current resume-replacement limitation can invalidate older comparisons; refreshing them is the existing recovery action.

Profile → Builder → Questions → User decision → Profile

Memory Agent

Retrieves the saved profile and its evidence



Purpose

Memory gives Profile access to the user’s authoritative saved information.

What it does

Loads records from Supabase and checks their current evidence and approval status. Storage revisions prevent an older update from overwriting newer information. Optional semantic retrieval uses Pinecone to find candidates, then checks those candidates against current records.

What it produces

The current profile, source references, retrieval results and storage receipts. The Python load_profile tool always reloads current information rather than reusing a cached profile response.

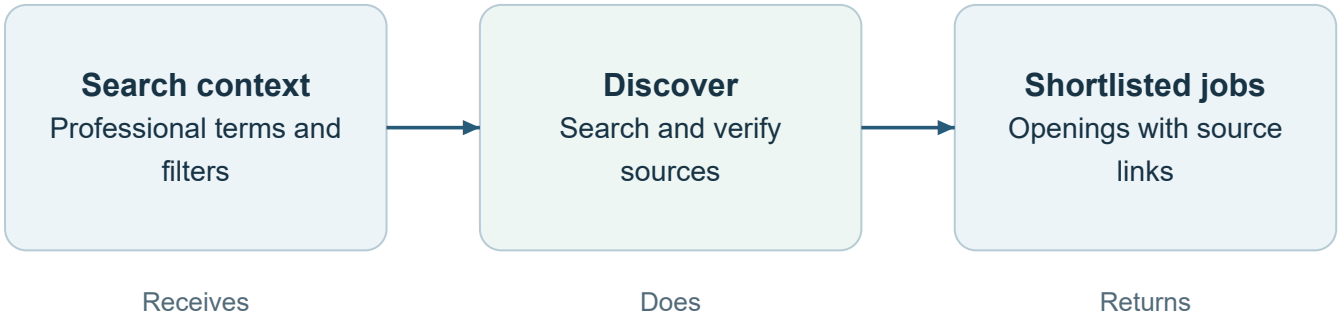
Who it works with

Only Profile coordinates Memory. Supabase is the source of truth. A Pinecone similarity score does not prove experience and cannot create an approval.

Profile ↔ Memory ↔ Supabase | Pinecone supports retrieval

Scout Agent

Finds real job openings on the web



Purpose

Scout turns the current professional profile and the user’s preferences into a search for actual vacancies.

What it does

Uses titles, responsibilities, industry and specialist skills to form bounded searches. Keenable retrieves results and source pages. Scout checks links, duplicates, dates, location and other filters, while keeping unknown information explicit.

What it produces

Source-linked jobs with employer, title, description, application URL, relevant experience and uncertainties. Gemini may prioritize supplied search terms; the vacancies themselves come from retrieved sources.

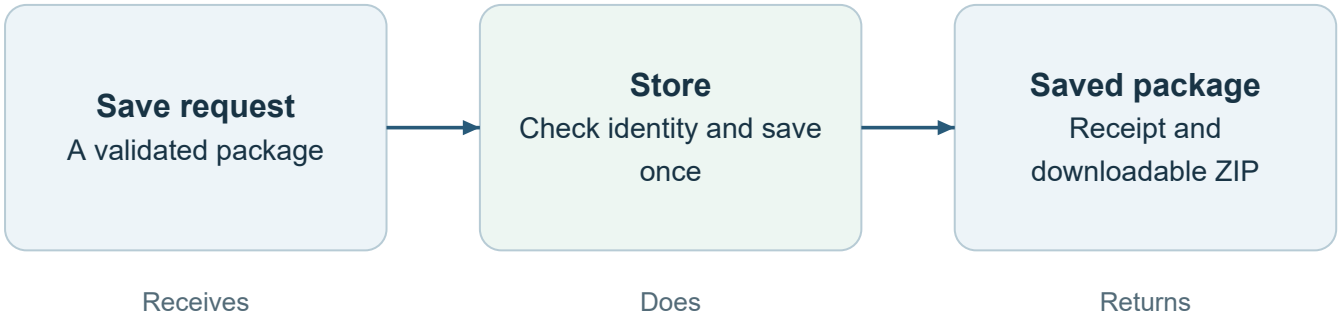
Who it works with

Receives scoped context through Master. Raw resume text and contact details are excluded from search queries. Opening the preview does not run a search; the user must start it.

Master → Scout → Keenable → Scout → Master

Application Storage

Keeps a validated application package



Purpose

Application Storage preserves a package that Master has already authorized for storage.

What it does

Checks package identity and validation references at the write boundary. Uses an idempotency key so replaying the same save does not create duplicates. Stores owner-scoped package data in the local SQLite database.

What it produces

A durable receipt tied to the job, profile version and package content. Saved packages can be downloaded as a ZIP and include the resume files, job details and validation records.

Who it works with

Receives save commands from Master. It does not edit the profile or submit an application. Packages remain on the local computer; they are separate from the Supabase profile.

Master → Application Storage → SQLite → Storage receipt

03 COMPLETE USER WORKFLOW

From resume to shortlisted jobs

The four website steps connect through one saved profile.

1 Add your resume

Upload or paste readable resume text.

Profile receives and stores the source.

2 Review facts and experience

Check the information extracted from your sources.

Fact returns evidence-linked factual proposals.

3 Expand experience and skills

Add optional JDs and answer relevant questions.

Builder compares requirements; Profile saves approved additions.

4 Search and shortlisted jobs

Review your filters and explicitly start a search.

Master loads the current profile; Scout returns actual openings.

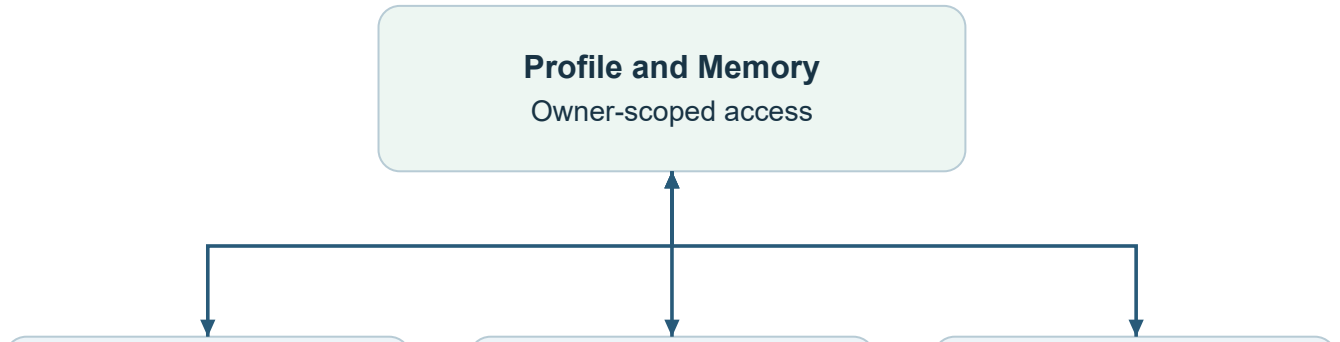
What continues between steps

The saved profile carries evidence, user decisions and search preferences forward. Search uses the current eligible profile; no separate Final Profile save is needed.

This is the four-step profile and discovery journey. Application Storage is a separate package-saving function.

Where the information is stored

Different stores have different responsibilities.



Profile facts, approvals, requirements and history

Original PDF, DOCX and text files

Searchable vectors derived from saved evidence

Local SQLite
Application runs and packages

Used by the local application runtime. It is separate from the saved profile.

Browser storage
Completed searches and bookmarks

Remembers search results on this browser. It is not cross-device profile storage.

The key distinction

Supabase is the source of truth. Pinecone helps find relevant evidence but cannot approve a fact. Local SQLite and browser storage hold separate application and search data.

What each external service does

Four services support reasoning, persistence and discovery.

Gemini
Model API

Understand and generate structured information

gemini-3.1-flash-lite supports analysis and comparison; gemini-embedding-001 creates search vectors.

Supabase
Database and file APIs

Keep the authoritative profile

Versioned records, approvals and private original files.

Pinecone
Vector API

Find semantically related evidence

Optional similarity search over vectors derived from saved records.

Keenable
Search and fetch APIs

Find and read actual job openings

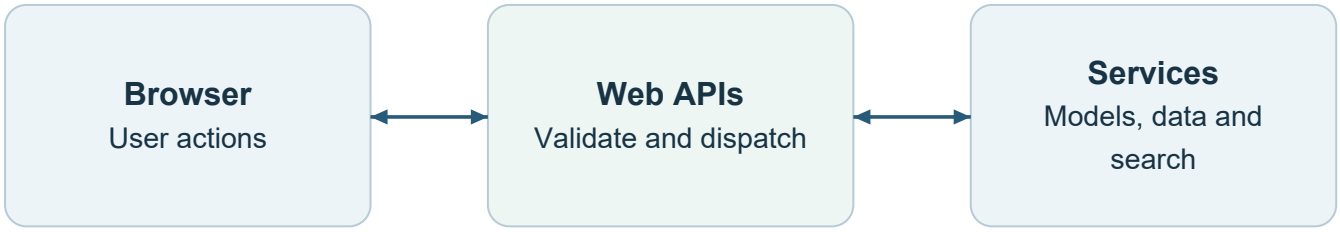
Public web search and source-page retrieval for vacancy checks.

How the services differ

Gemini interprets information. Supabase remembers it. Pinecone helps retrieve it. Keenable searches the public web for vacancies.

How the API calls connect

The browser uses the application server; secrets stay on the server.



Profile API

/api/profile handles source intake, factual decisions, Builder comparisons and profile retrieval. Supabase RPC and Storage APIs save the records and original files.

Search API

/api/scout previews profile-based search and starts discovery. Keenable's **/v1/search** finds pages and **/v1/fetch** reads source content.

Model and vector APIs

Gemini's **generateContent** returns structured model output. **batchEmbedContents** creates embeddings. Pinecone's **upsert**, **query** and **delete** operations maintain and search the derived index.

Local backend and package API

/api/pipeline connects the website to the local Python service. Python uses LangGraph and typed tools; a fixed Node bridge reuses the provider integrations. SQLite stores run history and packages.

Implementation: React and TypeScript interface; JavaScript Profile and Scout services; Python backend. Account identity comes from the trusted sign-in boundary. This guide describes the local implementation as of 10 November 2026.